

Week 3: Conditional statements and loops

What we discussed

- The value of a variable can be changed (reassigned).
- Assignments such as $i = i+1$ or $j = j*2$ are allowed.
 - They are useful for generating sequences
 - Once you can generate sequences, you can compute expressions such as $n!$
 - Other uses will be seen later
- Because assignments such as $i = i+1$ and $j = j*2$ are very frequently needed, operators such as $++$ and $*=$ have been provided.



Blocks and Scope

Code inside {} is called a **block**

- Blocks are associated with repeats, but you may create them otherwise too.
- You may declare variables inside any block.

New summing program:

- The variable `term` is defined close to where it is used, rather than at the beginning. This makes the program more readable.
- But the execution of this code is a bit involved.

```
// The summing program  
// written differently
```

```
main_program{  
    int s = 0;  
    repeat(10){  
        int term;  
        cin >> term;  
        s = s + term;  
    }  
    cout << s << endl;  
}
```

How definitions in a block execute

Basic rules

- A variable is defined/created every time control reaches the definition.
- All variables defined in a block are destroyed every time control reaches the end of the block.
- “Creating” a variable is only notional; the compiler simply starts using that region of memory from then on.
- Likewise “destroying” a variable is notional.
- New summing program executes exactly like the old, it just reads different (better!).

Scope and Shadowing

- Variables defined outside a block can be used inside the block, if no variable of the same name is defined inside the block.
- If a variable of the same name is defined, then from the point of definition to the end of the block, the newly defined variable gets used.
- The new variable is said to “**shadow**” the old variable.
- The region of the program where a variable defined in a particular definition can be used is said to be the **scope** of the definition.
- Why do we care:
 - In a single English language document you might write “Let x denote” in several places, with the understanding that the x on page 5 is different from the x on page 37.
 - If you do not have an intervening “Let x denote ..” the two xs might be same.
 - Something similar happens while writing large programs

Example

```
main_program{  
    int x=5;  
    cout << x << endl;    // prints 5  
    {  
        cout << x << endl; // prints 5  
        int x=10;  
        cout << x << endl; // prints 10  
    }  
    cout << x << endl; // prints 5  
}
```

What if `int x = 10;` was `x = 10;` ?

Concluding Remarks

- Variables are regions of memory which can store values.
- Variables have a type, as decided at the time of creation.
- Choose variable names to fit the purpose for which the variable is defined.
- The name of the variable may refer to the region of memory (if the name appears on the left hand side of an assignment), or its value (if the name appears on the right hand side of an assignment).

More remarks

- Expressions in C++ are similar to those in mathematics, except that values may get converted from integer to real or vice versa and truncation might happen.
- Truncation may also happen when values get stored into a variable.
- Sequence generation and accumulation are very common idioms.
- Increment/decrement operators and compound assignment operators also are commonly used.

More remarks

- Variables can be defined inside any block.
- Variables defined outside a block may get shadowed by variables defined inside.



Let us calculate income tax

Income Range	Tax percentage
0 to 4,00,000	0
4,00,001 to 8,00,000	5
8,00,001 to 12,00,000	10
12,00,001 to 16,00,000	15
16,00,001 to 20,00,000	20
20,00,001 to 24,00,000	25
24,00,001 and above	30

Cannot write tax calculation program using what you have learnt so far.

Outline

- Basic `if` statement
 - Program to solve a very simple tax problem
- `if-else` statement
 - Better program to solve the simple problem
- Most general `if` statement form
 - Full tax calculation program
- How to express complex conditions
- Case study: A different way to control the turtle
- The `switch` statement
 - Yet another way to control the turtle
- Logical data

Basic if statement

- Form:

`if (condition) consequent`

- `condition`: “Boolean” expression.
- “Boolean” : Should evaluate to “true” or “false”.
- `consequent`: C++ statement, e.g. assignment.
- `consequent` could also be a block, i.e. {...}
- If `condition` evaluates to true, then the `consequent` is executed.
- If `condition` evaluates to false, then `consequent` is ignored.

Conditions

Simple condition: `exp1 relop exp2`

- `relop` : relational operator:
 - `<` : less than. `<=` : less than or equal. `==` : equal.
 - `>` : greater than. `>=` : greater than or equal. `!=` : not equal
- Condition is considered true if `exp1` relates to `exp2` as per the specified relational operator `relop`.

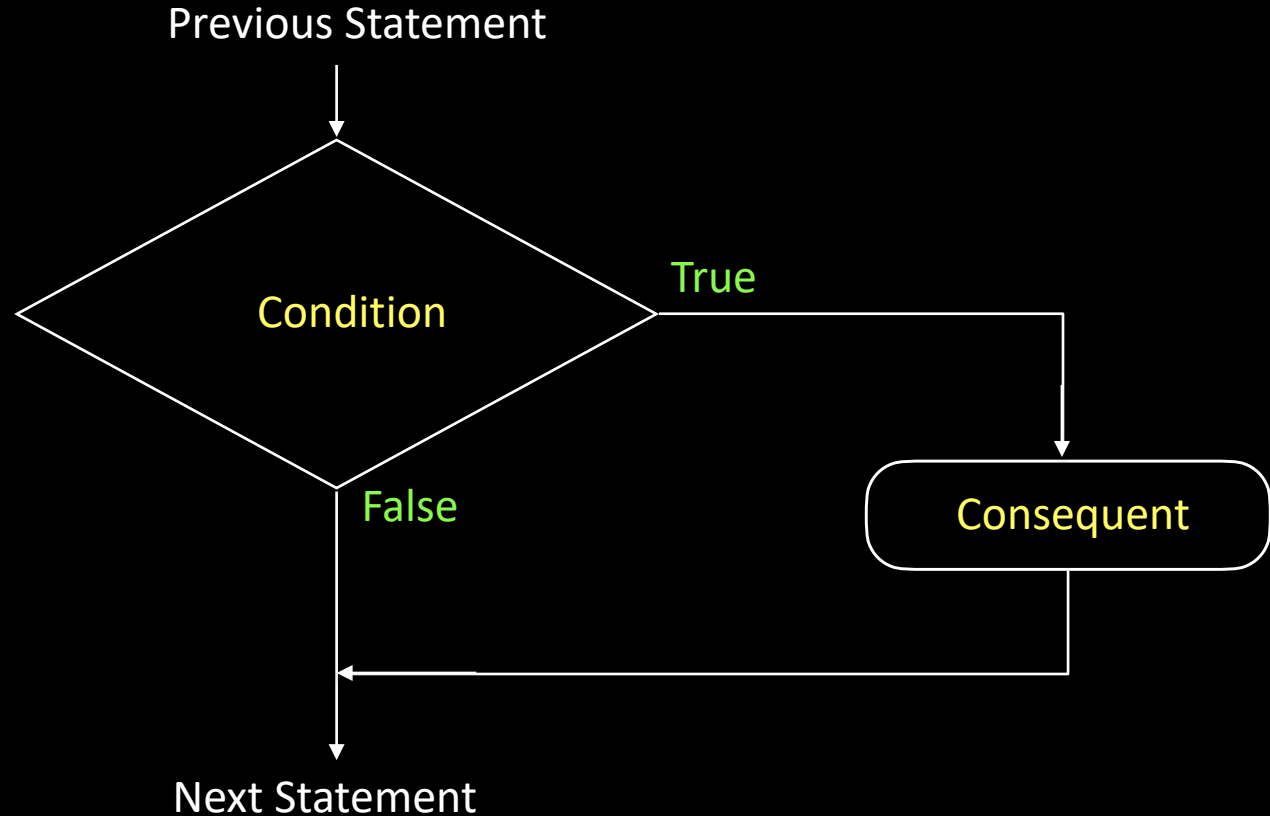
Suppose `x = 5`, `y = 10`, `z = 100`.

- `x >= y` is false.
- `x*x > y` is true.
- `x*y - z == 10` is false

Flowchart

- Pictorial representation of a program.
- Statements put inside boxes.
- If box C will possibly be executed after box B, then put an arrow from B to C.
- Specially convenient for showing conditional execution, because there can be more than one “next” statements.
- “Diamond” shaped boxes are used for condition checks.

Flowchart of `if(condition) consequent`



Simplified problem: just determine if any tax is owed

```
main_program{  
    float income, tax; cin >> income;  
    if(income <= 400000)  
        cout << "No tax owed." << endl;  
    if(income > 400000)  
        cout << "You owe tax." << endl;  
}
```

```
// Always checks both conditions.  
// If the first condition is true,  
// then you know second must be false,  
// and vice versa.  
// Can we avoid checking twice?
```

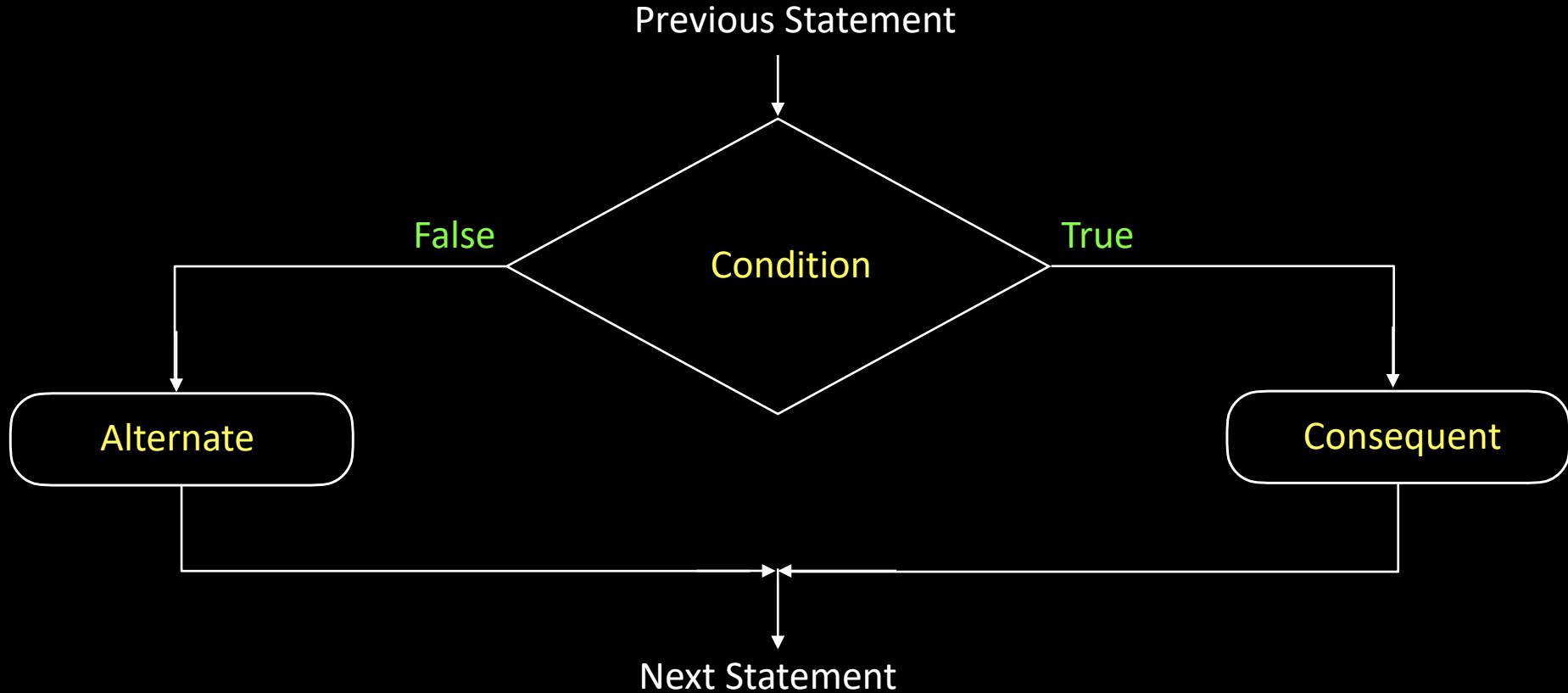

Another form of if

```
if (condition) consequent  
else alternate
```

- The `condition` is first evaluated.
- If it is true, then `consequent` is executed.
- If `condition` is false, then `alternate` is executed.

`alternate` can also be a block.

if else flowchart



Better program for simple problem

```
main_program{
    float income, tax; cin >> income;
    if(income <= 400000)
        cout << "No tax owed." << endl;
    else
        cout << "You owe tax." << endl;
}
// Only one condition check. Thus
// more efficient than previous.
```

Exercise

- Write a program that reads in a number and prints its square root. If the number is positive, it should use the `sqrt` function. If the number is negative, it should invoke `sqrt` on the negative of the number (which will be a positive quantity) and print the result followed by the letter 'i', to indicate that the result is imaginary.

What we discussed

- 2 Forms of the if statement.

Next: The most general form of the if statement.

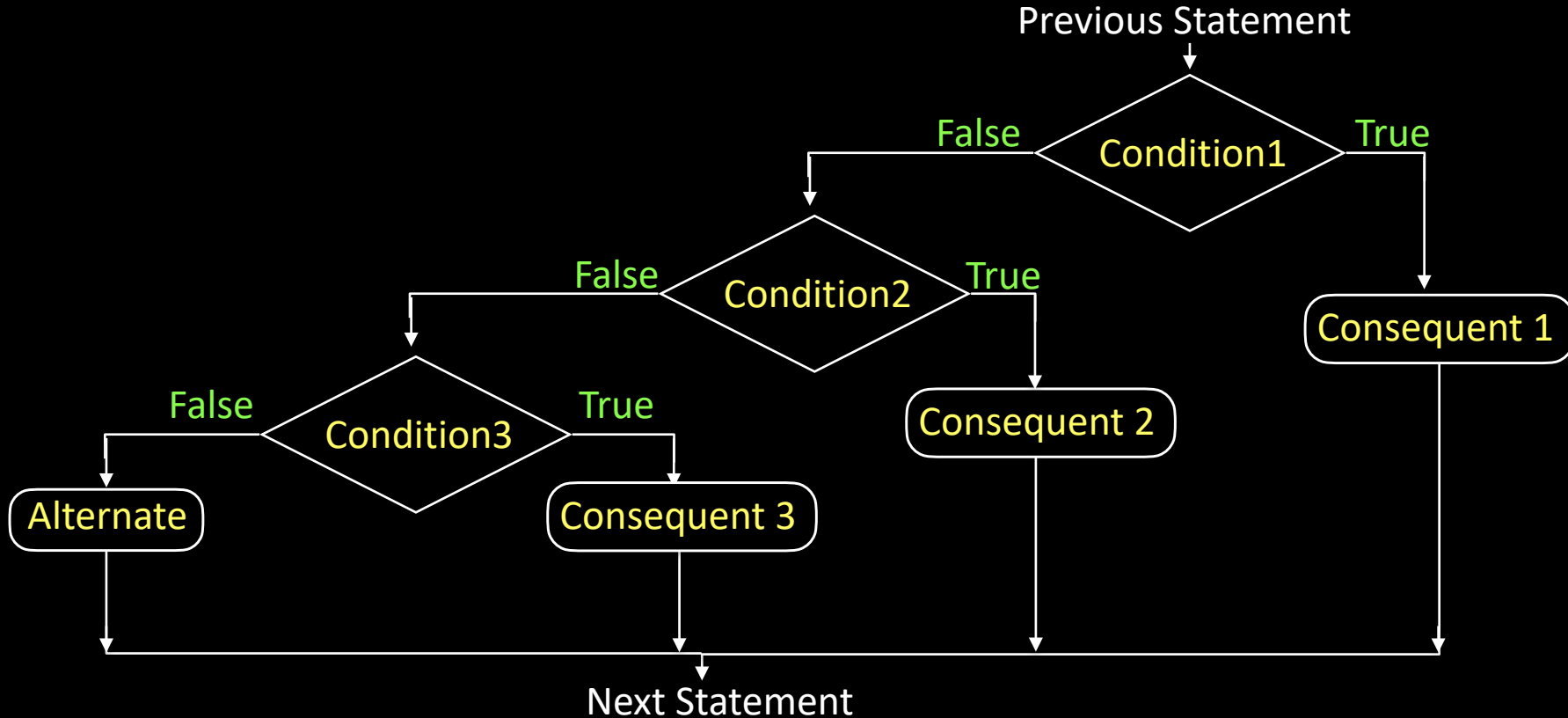


Most general form of if

```
if (condition1) consequent1  
else if (condition2) consequent2  
...  
else if (conditionn) consequentn  
else alternate          // optional
```

- Evaluate conditions in order.
- Some condition true: execute corresponding consequent. Do not evaluate subsequent conditions.
- All conditions false: execute alternate if specified.
- Consequents and alternate can be blocks or single statements.

General if example flowchart (with 3 conditions)



Tax calculation program

```
main_program{  
    float tax,income; cin >> income;  
    if (income <= 400000) tax = 0;  
    else if(income <= 800000)  
        tax = (income - 400000) * 0.05;  
    else if(income <= 1200000)  
        tax = (income - 800000) * 0.1 + 20000;  
        :  
    else  
        tax = (income-2400000)*0.3 + 300000;  
    cout << tax << endl;  
}
```

see the cpp file for complete program

Exercise: Is the following program correct? Precisely state the error, if any.

```
main_program{  
    float tax,income; cin >> income;  
    if (income <= 400000) tax = 0;  
    if(income <= 800000)  
        tax = (income - 400000) * 0.05;  
    if(income <= 1200000)  
        tax = (income - 800000) * 0.1 + 20000;  
        :  
    else  
        tax = (income-2400000)*0.3 + 300000;  
    cout << tax << endl;  
}
```

What we discussed

- Most general form of if statement
- Use in tax calculation program

Next: more general ways of specifying conditions



More general form of conditions

Sometimes we might want to do something if two conditions are true, or one of two conditions is true...

Compound conditions:

- “AND”: `condition1 && condition2` : true only if both true.
- “OR”: `condition1 || condition2` : true only if at least one is true.
- “NOT”: `! condition` : true if only if `condition` is false.

Components of compound conditions may themselves be compound conditions,

e.g. `condition1 && (condition2 || condition3)`

Example 1

```
if ((income >= 400000) && (income <= 800000))  
    tax = (income - 400000) * 0.05;
```

Example of execution

- `income = 100000` : Condition is false, consequent not executed.
- `income = 500000` :
 `tax = (500000 - 400000) * 0.05` is executed.
- `income = 900000` : Nothing happens.

Note

Same condition may be expressed in many ways.

`(income >= 400000)` is same as `!(income < 400000)`

`(income <= 800000)` is same as `!(income > 800000)`

Previous statement can be written as

```
if (!(income < 400000) && !(income > 800000))  
    tax = (income - 400000) * 0.05;
```

Another example

- Consider rectangle lying between lines $x=0$, $x=10$, $y=50$, $y=70$.
- Let (X,Y) denote the coordinates of a point.
- The point is inside the rectangle if $0 \leq X \leq 10$, and $50 \leq Y \leq 70$
- To check this we will write the condition:
 $(0 \leq X \ \&\& \ X \leq 10 \ \&\& \ Y \geq 50 \ \&\& \ Y \leq 70)$
- Do not write $0 \leq X \leq 10$

Another example

- Two rectangles of height h_1 , h_2 and width w_1 , w_2 .
- Rectangle 1 does not fit into Rectangle 2
 - If $h_1 > h_2$ or $w_1 > w_2$
- To check this we will write the condition:
 $(h_1 > h_2 \ || \ w_1 > w_2)$

Another example

- Three numbers, x , y , z
- x is the middle number if
- $(x < y \ \&\& \ x > z) \ || \ (x > y \ \&\& \ x < z)$

What we discussed

- More general ways of specifying the conditions.
- Note: ! Has higher precedence than && which has higher precedence than ||

$!P \ \&\& \ Q \ || \ R$ is same as $((!P) \ \&\& \ Q) \ || \ R$

Recommended: always use brackets

Next: A somewhat large example based on what we have learned so far.



A different way to control the turtle

- We will write a program which reads commands from the user, and accordingly controls the turtle.
 - ‘f’ : turtle should execute forward(100).
 - ‘r’ : turtle should turn right(90).
 - ‘l’ : turtle should turn left(90).
 - Stop after 100 commands are executed.

The program

```
main_program{
    char command;
    turtleSim();

    repeat(100){
        cin >> command;
        if (command == 'f') forward(100);
        else if (command == 'r') right(90);
        else if (command == 'l') left(90);
        else cout << "Not a proper command, "
                << command << endl;
    }
}
```

Exercise

- Write a program that reads a number and prints 1 if the number is a multiple of 5 but not of 3, and otherwise prints 0. Write this in as many different ways as possible.
 - Using only simple conditions, e.g. expression 1 == expression 2, but with if statements nested inside each other.
 - Using a single if-then-else statement with a compound condition.

What we discussed

- A detailed example

Next: The switch statement (will be discussed later if we have time) and logical data.



Logical Data

- We have seen that we can “evaluate” conditions, combine conditions.
- Why not allow storing the results (true or false) of such computations?
- Indeed, C++ has data type `bool` into which values of conditions can be stored.
- `bool` is named after George Bool, who formalized the manipulation of conditions/logical data.

The data type bool

```
bool highincome, lowincome;
```

- Defines variables `highincome` and `lowincome` of type `bool`.

```
highincome = (income > 5000000);
```

```
bool fun = true;
```

- Will set `highincome` to `true` if the variable `income` contains value larger than 5000000.
- `true` and `false` : boolean constants.
- boolean variables which have a value can be used wherever “conditions” are expected, e.g.

```
if(highincome) tax = ...
```

Exercise: write a program to test if a given number n is prime.

- How will you do this manually?
 - Eratosthenes' sieve
 - We are required to “remember” all the primes determined till n .
 - So far we have no way of doing this
- Can we do something less efficient, but without requiring us to remember too many things?
 - Check if any of the numbers from 2 to $n-1$ divide n .

Solution

```
#include <simplecpp>

main_program{
    int n, divisor=2; cin >> n;
    bool divisorFound = false; // no divisor found for n so far
    // check if divisor divides n as it varies from 2 to n-1
    // if divisor divides n, set divisorFound = true
    repeat(n-2){
        if(n % divisor == 0) divisorFound = true;
        divisor = divisor + 1;
    }
    if(!divisorFound) cout <<"Prime.\n";
    else cout <<"Composite.\n";
}
```

Exercise

Execute the program mentally for $n = 100$.

- What answer does it produce?
- Are you happy with how the program executes?

What we discussed

- Switch statement: Recommend do not use.
- Logical Data: Can store results of condition evaluation

Next: Lecture summary



Summary

- Conditional execution makes life interesting.
- 3 forms of if.
 - You can nest if statements inside each other: some pitfalls in this are discussed in the book.
- Compound conditions
- Logical data

Try the exercises at the end of the book.



Mark averaging

“Read marks of students from the keyboard and print the average.”

- Number of students not given explicitly.
- If a negative number is entered as mark, then it is a signal that all marks have been entered.
- Assume at least one positive number is given.

Examples:

- Input: 98 96 -1, Output: 97
- Input: 90 80 70 60 -1, Output: 75
- Cannot be done using what you know so far.
 - repeat repeats fixed number of times. Not useful

Need new statement e.g `while`, `do while`, or `for`

- `repeat`, `while`, `do while`, `for` : “looping statements”.

The while statement

Form `while (condition) body`

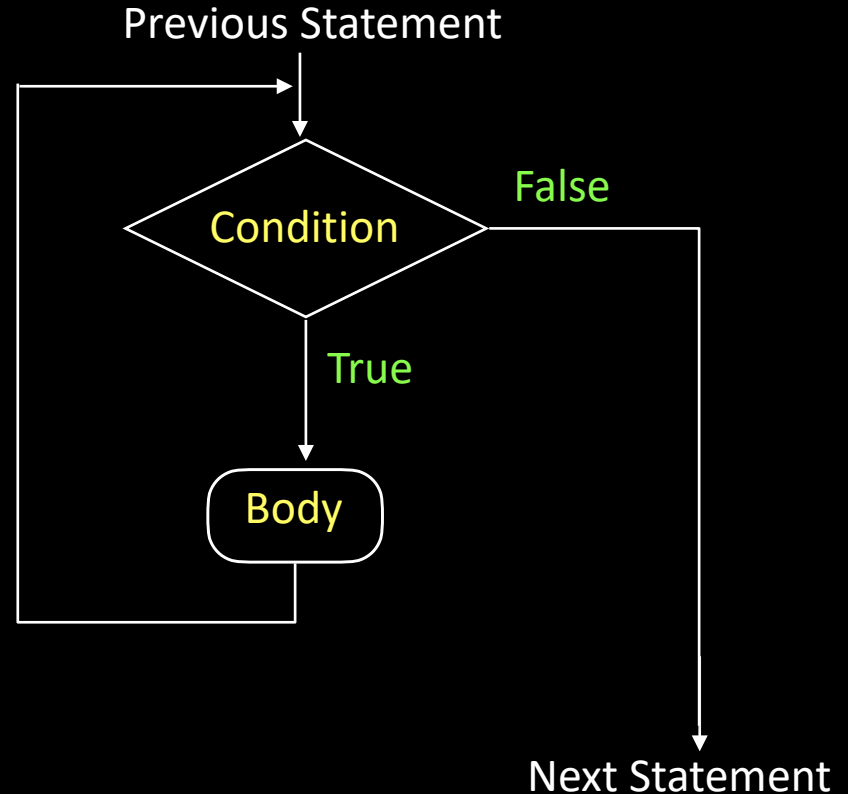
1. Evaluate `condition`.
 2. If `false`, execution of statement ends.
 3. If `true`, execute `body`. `body` can be a single statement or a block, in which case all the statements in the block will be executed.
 4. Go back and execute from step 1.
- The `condition` must eventually become `false`, otherwise the program will never halt. **Not halting is not acceptable.**
 - If `condition` is `true` originally, then value of some variable in `condition` must change in the execution of `body`, so that eventually `condition` becomes `false`.
 - Each execution of the `body` = **iteration**.

while flowchart

...previous statement...

while(condition) body

...Next statements...



A silly example

```
main_program{
    int x=2;
    while(x > 0){
        x--;
        cout << x << endl;
    }
    cout << "Done." << endl;
}
```

- First $x=2$ is executed.
- Next, $x > 0$ is checked
- $x=2$ is > 0 , so body entered.
- x is decremented, becomes 1.
- x is printed. (1)
- Back to top of loop.
- $x=1$ is > 0 , body entered.
- x is decremented, becomes 0.
- x is printed. (0)
- Back to top of loop.
- $x=0$ is not > 0 . body not entered.
- "Done." printed

while vs. repeat

- Anything you can do using repeat can be done using while.

```
repeat(n) { xxx }
```

- Equivalent to

```
int i=n;
```

```
while(i>0) { i--; xxx }
```

Assumption: the name *i* is not used elsewhere in the program.

- If it is, pick a different name

Exercise

- What will the following program fragment print?

```
int x=306, y=77, z=0;
while(x > 0){
    z = z + y;
    x--;
}
cout << z << endl;
```

- Write the following using while .

```
repeat(4){
    forward(100);
    right(90);
}
```

What we discussed

When to use `while` statement:

- We need to iterate while a condition holds
- We need not know at the beginning how many iterations are there.

Whatever we can do using `repeat` can be done using `while`

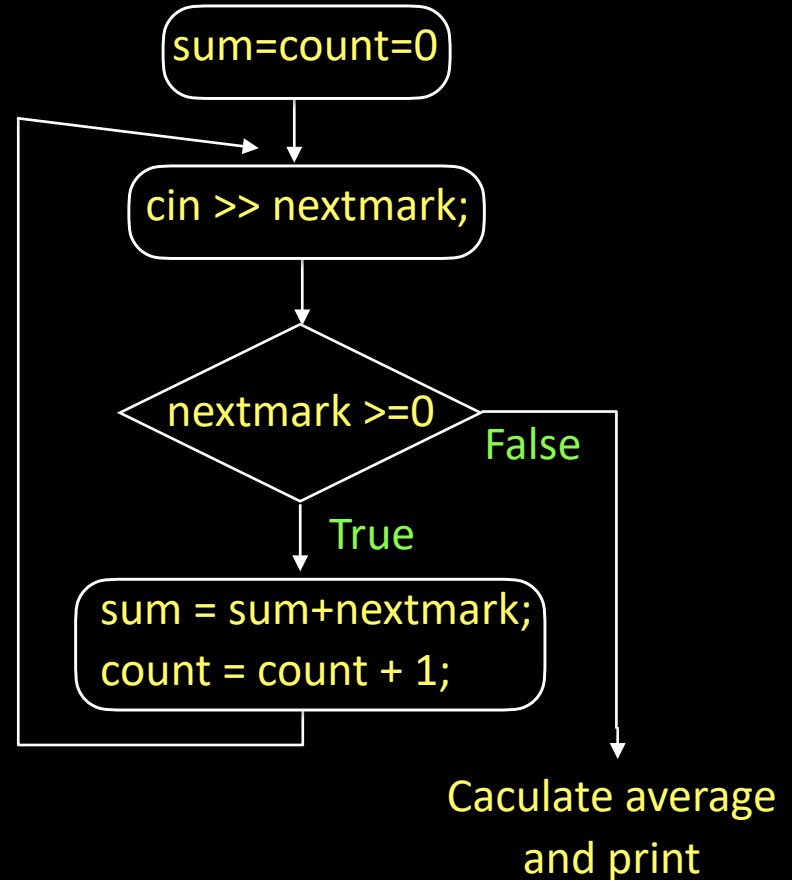
- Converse not true



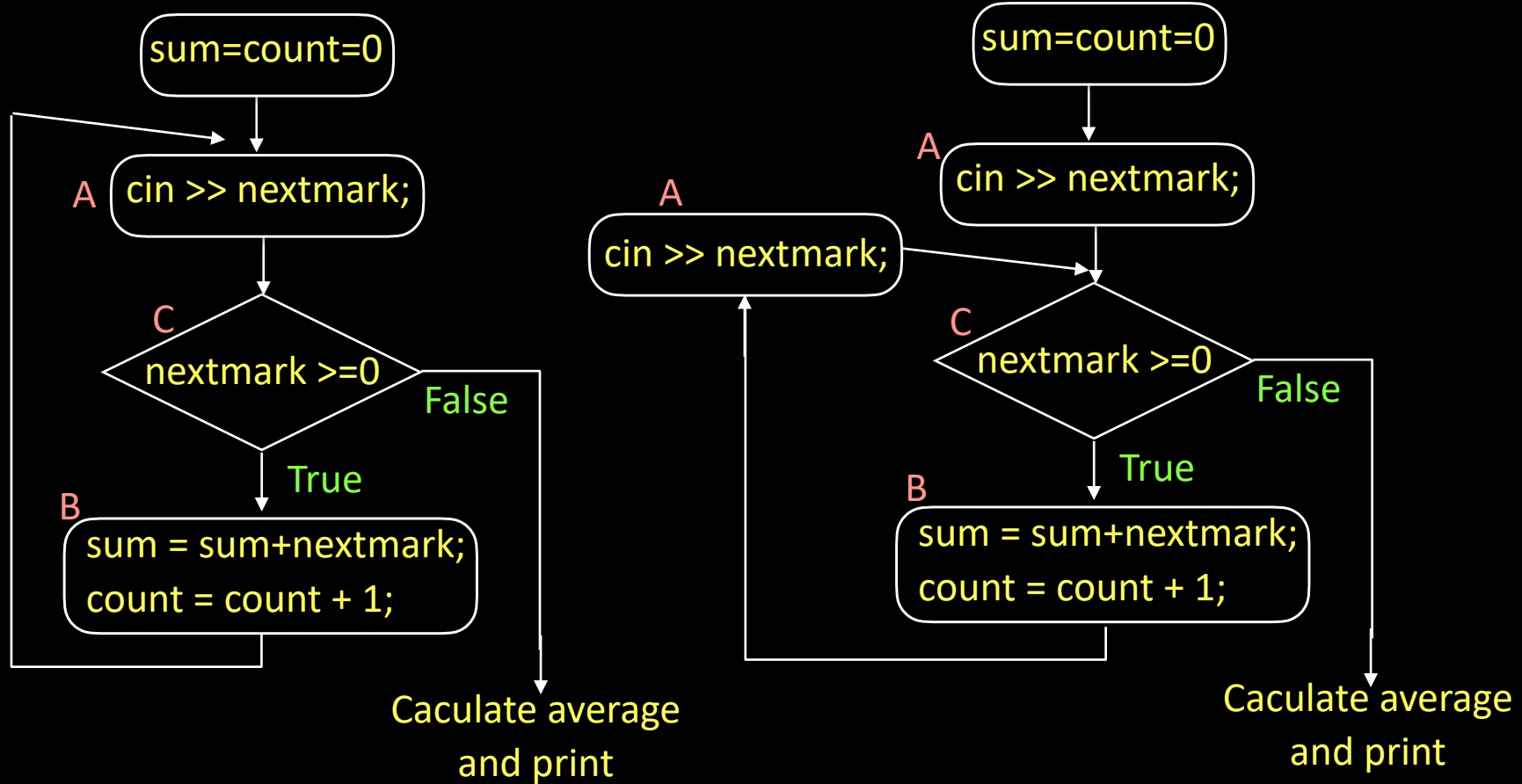
Mark averaging: manual algorithm

1. Set $\text{sum} = \text{count} = 0$.
2. Read next value into nextmark
3. If $\text{nextmark} < 0$, then go to step 7, if it is ≥ 0 , continue to step 4.
4. $\text{sum} = \text{sum} + \text{nextmark}$
5. $\text{count} = \text{count} + 1$
6. Go to step 2.
7. Print sum/count .

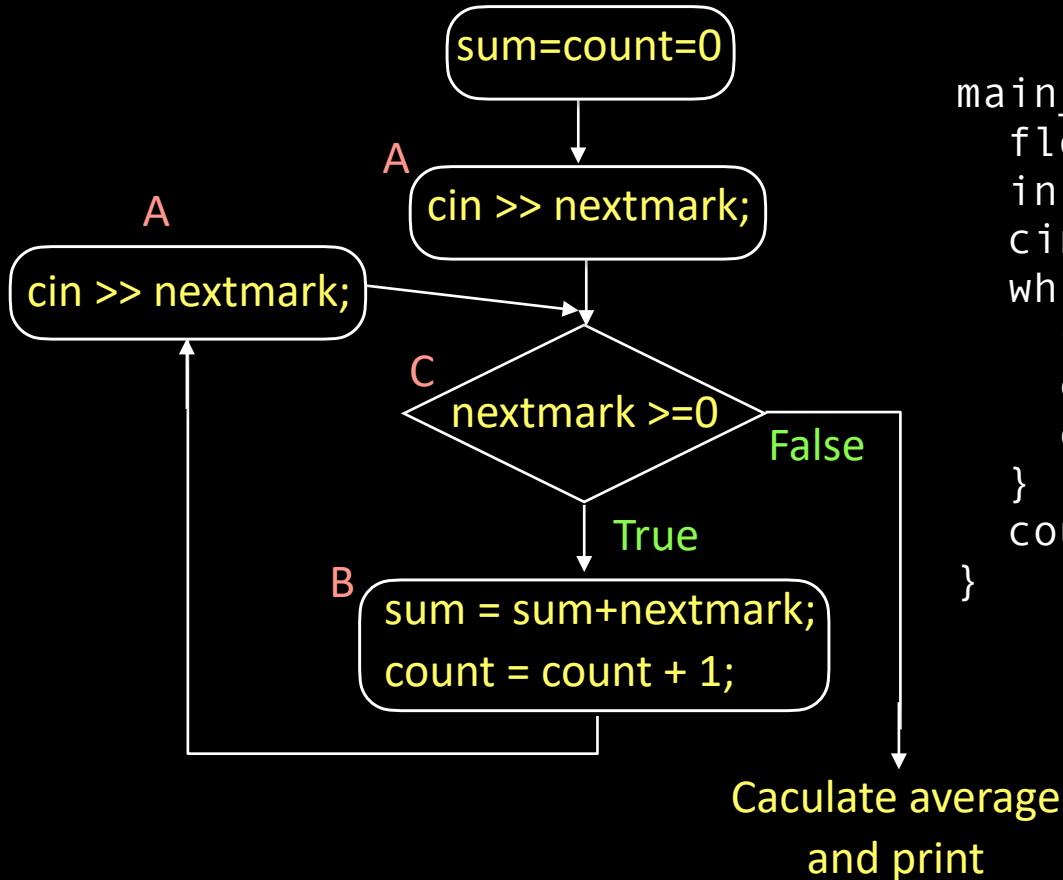
Structures does not match with while.



A different flowchart for mark averaging



A different flowchart for mark averaging



```
main_program{
    float nextmark, sum = 0;
    int count = 0;
    cin >> nextmark;           // A
    while(nextmark >= 0){      // C
        sum = sum + nextmark;  // B
        count = count + 1;     // B
        cin >> nextmark;       // A copy
    }
    cout << sum/count << endl;
}
```

Exercise

Write a turtle controller which receives commands from the keyboard and acts per the following rules:

- If command = 'f' : move turtle forward 100 pixels
- If command = 'r' : turn turtle right 90 degrees.
- If command = 'x' : finish execution.

What we discussed

- In while loops, the first step is to check whether we need to execute the next iteration.
- In some loops that we want to write, the first step is not a condition check. The condition check comes later.
- Such loops can be modified by making a copy of the steps before the condition check.
- NEXT: loops in which condition checks can happen also inside the body



The break statement

Form: The `break` keyword is a statement by itself.

What happens when control reaches break statement:

- The execution of the `while` statement which contains it is terminated.
- The execution continues from the next statement following that `while` statement.

Example of break

```
main_program{
    float nextmark, sum = 0;
    int count = 0;
    while(true){
        cin >> nextmark;
        if(nextmark < 0) break;
        sum += nextmark;
        count++;
    }
    cout << sum/count << endl;
}
```

- The condition of the while statement is given as true – body will always be entered.
- If nextmark < 0:
 - the while loop execution will terminate
 - Execution continues from the statement after while, i.e. cout ...
- Exactly what we wanted!
 - No need to copy code.
- Some programmers do not like break statements because continuation condition gets hidden inside body, instead of being at the top.
- Condition for breaking = compliment of condition for continuing loop

The continue statement

- The `continue` is another single word statement.
- If it is encountered in execution:
 - The control directly goes to the beginning of the loop for the next iteration,
 - Statements from the `continue` to the end of the loop body are skipped.

Example

Mark averaging with an additional condition:

- If a number > 100 is read, ignore it.
 - say because marks can only be at most 100
- Continue execution with the next number.
- As before stop and print the average only when a negative number is read.

Code for new mark averaging

```
main_program{
    float nextmark, sum = 0;
    int count = 0;
    while(true){
        cin >> nextmark;
        if(nextmark > 100) continue;
        if(nextmark < 0) break;
        sum += nextmark;
        count++;
    }
    cout << sum/count << endl;
}
```

The do while statement

- Not very common.
- Discussed in the book.

Exercise

- Write the turtle controller program using the break statement.

Requirements:

- 'f' : mover forward 100 pixels.
- 'r' : right turn 90 degrees.
- 'x' : exit program.

What we discussed

- The break statement enables us to exit a loop from inside the body.
- If break appears inside a while statement which is itself nested inside another while, then the inner while statement is terminated.
- The continue statement enables us to skip the rest of the iteration.

